Phase-Aware Hypergraph Distance – Final Version

This document compiles the **final production-ready phase-aware hypergraph distance** and provides an **exact patch set** to eliminate remaining silent-failure risks:

- Broken linear wiring in the generator
- Nonzero self-distance due to ambiguous Hungarian matchings on repeated identical actions
- Degenerate "deterministic" flow walks that always pick the first successor at step 0
- Non-deterministic action-signature ordering inside the distance computation

---

1) Final design goals

1.1 What "similar" means (aligned definition)

Two protocol hypergraphs are similar when they:

1. Have similar **action semantics** (types + parameters)
2. Have similar **phase placement** (phase distance under a grammar)
3. Have similar **connectivity/topology** (branching/merging structure)
4. Respect similar **phase progression along edges** (process logic)
5. Exhibit similar **global phase-flow patterns** (path/trigram statistics)

1.2 Output requirements

- Deterministic: `d(G,H)` is reproducible
- Proper baseline: `d(G,G) = 0`
- Symmetric: `d(G,H) = d(H,G)`
- Robust under repeated steps: repeated identical actions should not introduce arbitrary matching artifacts

---

2) Final distance definition

Let graphs be `G1, G2`.

2.1 Assignment-based node matching (Hungarian)

Compute a padded cost matrix `C` between action-signatures of `G1` and `G2` plus dummy nodes.

- Real–real costs: semantic + phase + structure + neighborhood + parameter
- Real–dummy costs: deletion
- Dummy–real costs: insertion
- Dummy–dummy costs: 0

The Hungarian assignment yields a minimum-cost bijection.

2.2 Topology mismatch

Using the induced mapping from matched action IDs:

$$\mathrm{Topo}(G_1, G_2) = \frac{|E_1^{\backslash \mathrm{map}} \triangle E_2|}{|E_1^{\backslash \mathrm{map}} \cup E_2|}$$

2.3 Phase progression mismatch

Compare progression penalties along matched edges between the two graphs:

$$\mathrm{Prog}(G_1, G_2) = \mathrm{avg}_{(u \to v) \in E_1^{\backslash \mathrm{map}}} \left| p_1(u \to v) - p_2(\backslash\mathrm{map}(u) \to \backslash\mathrm{map}(v)) \right|$$

Where `p(·)` is derived from the phase grammar.

2.4 Flow mismatch (deterministic)

Compute deterministic phase sequences from a seed derived from the graph's canonical hash, then compare phase trigrams with Jaccard dissimilarity.

$$\mathrm{Flow}(G_1, G_2) = 1 - \frac{|T_1 \cap T_2|}{|T_1 \cup T_2|}$$

2.5 Final normalized distance

$$d(G_1,G_2)=\frac{\text{Assign}(G_1,G_2)+w_{\mathrm{topo}}\,\text{Topo}+w_{\mathrm{prog}}\,\text{Prog}+w_{\mathrm{flow}}\,\text{Flow}}{\max(|A_1|+|A_2|)/2,1)}$$

---

3) Production invariants (must hold)

1. **Wiring invariant:** For each state `s`,
2. `s.produced_by` is an action id or `None` (source state)
3. `s.consumed_by` contains all action ids that consume it
4. **Action adjacency:** an edge `a→b` exists iff some state is output of `a` and input to `b`
5. **Canonical hash includes wiring:** stable under ID renaming
6. **Distance idempotence:** `d(G,G)=0`

---

4) Exact patch set

The following patch is written against the code you posted.

Patch A — Fix generator linear wiring (critical)

Problem: `generate_linear()` creates new input states instead of consuming the previous output.

Replace inside `ProductionProtocolGenerator.generate_linear()`:

```
# Create input state if not first action
if prev_state is not None:
    input_state = self._create_state()
    G.add_state(input_state)
    action.input_states.append(input_state.id)
```

With:

```
# Consume previous output state (true linear chain)
if prev_state is not None:
    action.input_states.append(prev_state)
```

(No new input state is created.)

Patch B — Guarantee d(G,G)=0 via canonical-hash short-circuit

Problem: Hungarian can pick a different optimal permutation when actions repeat, causing nonzero topology/progression even when graphs are identical.

Add at top of `PhaseAwareHypergraphDistance.compute_distance()`:

```
# Fast path: canonical identity implies zero distance
if G1.get_canonical_hash() == G2.get_canonical_hash():
    return 0.0
```

This is correct relative to the canonicalization semantics already used by caching.

Patch C — Deterministic but non-degenerate flow walks

Problem: successor selection uses `(walk_idx * step * seed)`, which forces the first hop to always choose successor[0] when `step == 0`.

In `_get_deterministic_phase_sequences()`, **replace** the deterministic index arithmetic with seeded RNG draws.

Replace:

```
start_idx = (walk_idx * seed) % len(sources)
current = sources[start_idx]
...
succ_idx = (walk_idx * step * seed) % len(successors)
current = successors[succ_idx]
```

With:

```
# Deterministic RNG seeded by canonical hash
current = sources[rng.randint(len(sources))]
...
current = successors[rng.randint(len(successors))]
```

Because `rng` is seeded by canonical hash, this remains reproducible.

Patch D — Deterministic action-signature ordering inside distance computation

Problem: `_extract_action_signatures()` iterates `G.actions.items()` which can differ across merges or insertion order, and can change Hungarian outcomes.

Fix: sort actions by a deterministic signature before producing `acts1/acts2`.

In `_extract_action_signatures()`, build a list of action IDs and sort them:

```
action_ids = list(G.actions.keys())
# Deterministic order
action_ids.sort(key=lambda aid: (
    G.actions[aid].type,
    (G.actions[aid].phase.value if G.actions[aid].phase else ""),
    tuple(sorted(G.actions[aid].parameters.items()))
))

for action_id in action_ids:
    action = G.actions[action_id]
    ...
```

This makes signature lists stable.

---

5) Optional hardening (recommended)

5.1 Enforce single-producer invariant

In `ProtocolGraph._connect_action()` you currently allow multiple producers by silently passing.

Recommended production behavior:

- default: raise error (invalid graph)
- optional: allow if you explicitly support state fusion

Suggested guarded code:

```
if state.produced_by is None:
    state.produced_by = action.id
elif state.produced_by != action.id:
    raise ValueError("State has multiple producers; invalid unless explicitly supported")
```

---

6) Expected outcomes (predictions)

After applying patches A–D:

- **Self-distance:** will be exactly 0 for identical graphs
- **Symmetry:** will hold to floating precision
- **Branch sensitivity:** linear vs branched distance increases consistently
- **Merge sensitivity:** merged protocols become detectably farther than linear baselines
- **Flow term:** deterministic and diverse enough to discriminate different global process logics
- **Caching:** canonical hash yields correct exact hits; distance becomes stable across runs

---

7) Fastest path to validation

1. Apply patches A–D.
2. Re-run `run_comprehensive_validation()` with 100 randomized trials:
3. ensure `self_distance_zero` never fails
4. ensure symmetry difference stays < 1e-9
5. Stress test repeated actions:
6. build protocol with 5 identical `wash` actions; verify `d(G,G)=0`
7. Stress test merge order dependence:
8. merge chains in swapped order; verify canonical hash equality (if structurally identical)

---

8) Where this goes next (recommended)

Metric learning for weights

Expose a feature vector:

- type_cost
- phase_cost
- param_cost
- structure_cost
- topology_cost
- progression_cost
- flow_cost

Then learn nonnegative weights with triplet loss on (anchor, positive, negative) protocol triplets.

This yields data-driven alignment while keeping interpretability.

References

[R1] H. Bunke, “On a relation between graph edit distance and maximum common subgraph,” *Pattern Recognition Letters* 18(8), 1997.

[R2] K. Riesen and H. Bunke, *Graph Classification and Clustering Based on Vector Space Embedding*, World Scientific, 2009. (Background on graph edit distance approximations and assignment-based methods.)

[R3] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly* 2(1–2), 1955.

[R4] J. Munkres, “Algorithms for the assignment and transportation problems,” *Journal of the Society for Industrial and Applied Mathematics* 5(1), 1957.

[R5] R. W. Floyd, “Algorithm 97: Shortest Path,” *Communications of the ACM* 5(6), 1962.

[R6] S. Warshall, “A theorem on Boolean matrices,” *Journal of the ACM* 9(1), 1962. (All-pairs reachability / path closure; commonly paired with Floyd for APSP derivations.)

[R7] P. Jaccard, “Étude comparative de la distribution florale dans une portion des Alpes et du Jura,” *Bulletin de la Société Vaudoise des Sciences Naturelles* 37, 1901. (Jaccard similarity.)

[R8] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006. (Finite-state / grammar models for phase constraints.)

[R9] A. Doucet, N. de Freitas, and N. Gordon (eds.), *Sequential Monte Carlo Methods in Practice*, Springer, 2001. (Context for SMC/particle rejuvenation when integrating distance into filtering.)